Correction — TP 4

Module : Bases de Données
Niveau : L2
Responsable du TP : M. Alachaher
Année universitaire : 2025/2026

Partie 1 — Les Jointures (JOIN)

Exercice 1 : Jointures simples

```
-- Livres avec leur éditeur
SELECT L.titre, E.nom
FROM Livre L
JOIN Editeur E ON L.id_editeur = E.id_editeur;

-- Livres avec auteur
SELECT L.titre, L.prix, L.disponible, A.nom
FROM Livre L
JOIN Auteur A ON L.id_auteur = A.id_auteur;

-- Ventes avec titre et prix
SELECT V.id_vente, L.titre, L.prix
FROM Vente V
JOIN Livre L ON V.id_livre = L.id_livre;
```

Exercice 2 : Jointures multiples

```
-- Vente + employé
SELECT V.id_vente, Emp.nom
FROM Vente V
JOIN Employee Emp ON V.id_employee = Emp.id_employee;

-- Vente + livre + adhérent
SELECT L.titre, Ad.nom
FROM Vente V
JOIN Livre L ON V.id_livre = L.id_livre
JOIN Adherent Ad ON V.id_adherent = Ad.id_adherent;
```

```
-- Jointure complète
SELECT L.titre, A.nom AS auteur,Ed.nom As Editeur, Ad.nom AS adherent, Emp.nom AS empl
FROM Vente V
JOIN Livre L ON V.id_livre = L.id_livre
JOIN Auteur A ON L.id_auteur = A.id_auteur
JOIN Editeur Ed ON L.id_editeur = Ed.id_editeur
JOIN Adherent Ad ON V.id_adherent = Ad.id_adherent
JOIN Employee Emp ON V.id_employee = Emp.id_employee;
```

Exercice 3 : Jointures avec conditions

```
-- Livres > 1000
SELECT L.titre, A.nom
FROM Livre L
JOIN Auteur A ON L.id_auteur = A.id_auteur
WHERE L.prix > 1000;

-- Ventes > 1500 (prix du livre)
SELECT V.id_vente, L.prix
FROM Vente V
JOIN Livre L ON V.id_livre = L.id_livre
WHERE L.prix > 1500;

-- Livres non disponibles
SELECT L.titre, E.nom
FROM Livre L
JOIN Editeur E ON L.id_editeur = E.id_editeur
WHERE L.disponible = false;

-- Ventes de cette anneé
SELECT V.*, A.nom
FROM Vente V
JOIN Adherent A ON V.id_adherent = A.id_adherent
WHERE EXTRACT( YEAR FROM A.date_inscription) = 2026;
```

Exercice 4 : Jointures avec agrégation

```
-- Nombre de livres par auteur
SELECT A.id_auteur, A.nom, COUNT(L.id_livre) AS Nombre_livres
FROM Auteur A
JOIN Livre L ON A.id_auteur = L.id_auteur
```

```
GROUP BY A.id_auteur;

-- Total ventes par livre
SELECT L.titre, COUNT(V.id_vente)
FROM Livre L
JOIN Vente V ON L.id_livre = V.id_livre
GROUP BY L.titre;

-- Chiffre d'affaire par employé
SELECT Emp.nom, SUM(L.prix)
FROM Vente V
JOIN Livre L ON V.id_livre = L.id_livre
JOIN Employee Emp ON V.id_employee = Emp.id_employee
GROUP BY Emp.nom;

-- Nombre d'achat par adhérent
SELECT Ad.nom, COUNT(V.id_vente)
FROM Adherent Ad
JOIN Vente V ON Ad.id_adherent = V.id_adherent
GROUP BY Ad.nom;
```

Exercice 5 : LEFT JOIN

```
-- Tous les auteurs
SELECT A.nom, L.titre
FROM Auteur A
LEFT JOIN Livre L ON A.id_auteur = L.id_auteur;

-- Tous les livres
SELECT L.titre, V.id_vente
FROM Livre L
LEFT JOIN Vente V ON L.id_livre = V.id_livre;

-- Tous les adhérents
SELECT Ad.nom, V.id_vente
FROM Adherent Ad
LEFT JOIN Vente V ON Ad.id_adherent = V.id_adherent;

-- Tous les employés
SELECT Emp.nom, V.id_vente
FROM Employee Emp
```

```
LEFT JOIN Vente V ON Emp.id_employee = V.id_employee;
```

Partie 2 — Requêtes imbriquées

Exercice 1 : Sous-requêtes simples

```
-- Livres plus chers que la moyenne
SELECT *
FROM Livre
WHERE prix > (SELECT AVG(prix) FROM Livre);

-- Adhérents après id=5
SELECT *
FROM Adherent
WHERE date_inscription > (
    SELECT date_inscription
    FROM Adherent
    WHERE id_adherent = 5
);

-- Employés >= moyenne salaire
SELECT *
FROM Employee
WHERE salaire >= (SELECT AVG(salaire) FROM Employee);
```

Exercice 2 : Sous-requêtes avec IN

```
-- Livres des auteurs en C
SELECT *
FROM Livre
WHERE id_auteur IN (
    SELECT id_auteur FROM Auteur WHERE nom LIKE 'C%'
);

-- Ventes de livres >1800
SELECT *
FROM Vente
WHERE id_livre IN (
    SELECT id_livre FROM Livre WHERE prix > 1800
);
```

```
-- date inscription = 2025
SELECT *
FROM vente
WHERE id_adherent IN (
    SELECT id_adherent
    FROM adherent
 WHERE EXTRACT(YEAR FROM date_inscription)=2025
);
```

Exercice 3 : Sous-requêtes avec agrégations

```
-- Livres > moyenne des non disponibles
SELECT *
FROM Livre
WHERE prix > (
    SELECT AVG(prix)
    FROM Livre
    WHERE disponible = false
);

-- Employé le moins payé
SELECT *
FROM employee
WHERE salaire = (
    SELECT MIN(salaire)
    FROM employee
);
-- prix > somme des prix de l'auteur 2
SELECT *
FROM livre
WHERE prix >= (
    SELECT SUM(prix)
    FROM livre
 where id_auteur=2
);
```

Exercice 4 : Sous-requêtes avancées

```
-- Livres jamais vendus
```

```
SELECT *
FROM livre
WHERE id_livre NOT IN(
    SELECT DISTINCT id_livre
    FROM vente
 GROUP BY id_livre
);

-- Auteurs sans livre
SELECT *
FROM Auteur
WHERE id_auteur NOT IN (
    SELECT DISTINCT id_auteur
    FROM Livre
 GROUP BY id_auteur
);


-- Auteurs avec >2 livres
SELECT *
FROM Livre
WHERE id_auteur IN (
    SELECT id_auteur
    FROM Livre
    GROUP BY id_auteur
    HAVING COUNT(*) > 2
);

-- Auteur avec plus de livres
SELECT nom
FROM Auteur
WHERE id_auteur = (
    SELECT id_auteur
    FROM Livre
    GROUP BY id_auteur
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

Exercice 5 : Réflexion

```
-- JOIN vers sous-requête
SELECT titre
FROM Livre
WHERE id_auteur IN (
    SELECT id_auteur FROM Auteur WHERE nom = 'Hugo'
);

-- Sous-requête vers JOIN
SELECT L.titre
FROM Livre L
JOIN Auteur A ON L.id_auteur = A.id_auteur
WHERE A.nom = 'Hugo';
```

- JOIN : plus rapide et lisible pour relations
- Sous-requête : utile pour filtrage complexe

Remarque

JOIN est préférable pour les relations entre tables. Les sous-requêtes sont utiles pour les calculs ou filtres avancés.